

## Session 5 — Build Your AI Game (Student)

This is the final build day!

You already made:

- an AI **brain** that recognizes images
- an AI **personality** that talks to the player

Today, you will connect everything into one game.

### Class Plan (about 2 hours)

1. Review the full project
2. Load saved settings
3. Load game images
4. Build the game loop
5. Add score and AI reactions
6. Play and improve your game

### Game Flow

Show image → Player guesses → AI checks → Score updates → AI talks → Next round

## Grab the Toolboxes

```
In [ ]: import json
import random
from pathlib import Path

import matplotlib.pyplot as plt
from PIL import Image
```

- import json: Bringing in the tool to read the "recipe book" (data).
- import random: Bringing in a pair of dice (for making the game unpredictable).
- from PIL import Image: Putting on our glasses so we can actually see the pictures.

- pathlib: Grabbing a map so we don't get lost looking for folders.

## The Big Concept: "The Game's Memory"

Imagine you turn off your PlayStation or Xbox. When you turn it back on, it remembers your high score and your character's name. This code is the Loading Screen that checks the game's memory file (game\_settings.json) to see what kind of game it's supposed to be today.

```
In [ ]: settings_path = Path("session_data/game_settings.json")
if settings_path.exists():
    with open(settings_path, "r", encoding="utf-8") as f:
        settings = json.load(f)
else:
    settings = {
        "classes": ["cat", "dog"],
        "personality": "friendly",
        "response_bank": {},
        "hint_bank": {}
    }

classes = settings["classes"]
personality = settings.get("personality", "friendly")
response_bank = settings.get("response_bank", {})
hint_bank = settings.get("hint_bank", {})

print("Labels:", classes)
print("Personality:", personality)
```

### The "Safety Net" (The if/else block):

- "What happens if our memory file is accidentally deleted? The game shouldn't just crash! The else part is our backup plan. It says: 'If you can't find the file, just start a basic Cat vs. Dog game.'"

### The classes Variable:

- This is the "brain" of the game. It's a list of everything the AI is allowed to recognize. If you want to build a "Dragon vs. Knight" game, this is exactly where you will change those names later.

### The personality Variable:

- This is the "vibe" of the AI. Is it a friendly teacher or a sassy robot?
- The .get() part. It's a "polite" way of asking for information. It says: "Try to find the personality, but if it's not there, just be 'friendly' by default."

## Step 1 — Load Images

## Step-by-Step Logic:

- The Goal: Find the folder.
- The Search: Look for specific "tags" (.png, .jpg).
- The Result: Count them and show the first few.

```
In [ ]: # 1. Tell the code where to look
asset_folder = Path("game_assets")
image_paths = []

print("🔍 Scanning for game assets...")
```

```
In [ ]: # 2. Search for all types of image files
if asset_folder.exists():
    # Look for all common image types
    for ext in ["*.png", "*.jpg", "*.jpeg", "*.webp"]:
        image_paths.extend(asset_folder.glob(ext))

    # Sort them so they are always in a predictable order
    image_paths = sorted(image_paths)

    # 3. Give the student a clear status update
    count = len(image_paths)
    if count > 0:
        print(f"✅ SUCCESS! I found {count} images.")

        # --- THE PREVIEW POP-UP ---
        print("🖼️ Testing your first asset...")
        sample_img = Image.open(image_paths[0])
        plt.figure(figsize=(3, 3))
        plt.imshow(sample_img)
        plt.title(f"Loaded: {image_paths[0].name}")
        plt.axis("off")
        plt.show()
    else:
        print(f"⚠️ WARNING: '{asset_folder}' is empty! Please upload images.")
else:
    print(f"❌ ERROR: Folder '{asset_folder}' not found.")
    print("👉 Action: Click the folder icon on the left and create 'game_assets'!")
```

- The Scanner: That glob(ext) is like a search bar. It's looking for anything that ends in .png or .jpg.
- The [0] index: That image\_paths[0] means "the very first one." In coding, we start counting at zero, not one!
- The Proof: If you can see your image here, your game loop is officially ready to run. You've successfully connected your computer's files to your Python code!"

- How to use glob to count how many "cats" specifically are in their folder?

## Step 2 — The "Game Skills" (Helper Functions)

To keep this final class smooth, we use a simple label rule based on the file name.

Example:

- `cat_01.png` → answer is `cat`
- `dog_02.jpg` → answer is `dog`

If you want, you can replace this later with your real classifier.

Right now, your computer is like a baby. It has no idea how to play a game. We are using `def` (which stands for Define) to teach it 4 specific skills so it can be our Game Master.

### 1. `show_image` — The Game's Eyes 🦋

- What it does: It takes a file path and turns it into a picture on the screen.
- Without this, our game is just a boring text adventure. This skill tells the computer: 'Open the file, take off the ugly axis numbers (like a ruler), and show the player what they're looking at!'
- **Pay attention to:**

`plt.axis("off")`. What would happen if we turned this 'on'?" (Answer: It would look like a math graph with X and Y numbers!)

```
In [ ]: def show_image(image_path):  
        image = Image.open(image_path).convert("RGB")  
        plt.figure(figsize=(4, 4))  
        plt.imshow(image)  
        plt.axis("off")  
        plt.show()
```

### 2. `answer_from_filename` — The Secret Decoder 🕵️

- What it does: It looks at the filename (like `cat_01.jpg`) and guesses the answer is "cat."
- This is a 'cheat code'. We tell the computer: 'Look at the name of the file. If the word "dog" is hidden inside the name, then the answer must be dog!'
- **Pay attention to:**

`image_path.stem`. Stem is just the name of the file without the `.jpg` part.

```
In [ ]: def answer_from_filename(image_path):
        name = image_path.stem.lower()
        for label in classes:
            if label.lower() in name:
                return label
        return classes[0]
```

### 3. game\_response — The AI's Mouth 🗨️

- What it does: It picks a random sentence for the AI to say.
- This is where your AI's Personality comes to life. If the player is right, the AI shouldn't just say 'Correct' every time—that's robotic. This skill lets the AI reach into its 'Response Bank' and pull out a random high-five or a sassy comment.
- Pay attention to:

```
random.choice. This is the most important part for fun—it makes the
game feel different every time you play.
```

```
In [ ]: def game_response(correct):
        if response_bank and personality in response_bank:
            mode = "correct" if correct else "wrong"
            return random.choice(response_bank[personality][mode])
        return "Nice!" if correct else "Try again!"
```

### 4. hint\_for\_label — The Helpful Friend 💡

- What it does: If the player is stuck, it gives a specific hint for that animal/item.
- If the player is struggling with a picture of a 'Golden Retriever,' this skill looks into your 'Hint Bank' and says: 'Hey, try giving them a hint about floppy ears!'

```
In [ ]: def hint_for_label(label):
        return hint_bank.get(label, "Look carefully at the picture.")
```

Right now, our `answer_from_filename` is 'cheating' by looking at the name. Next class, we are going to replace this 'cheat' with a Real AI Brain that actually looks at the pixels. Which do you think will be faster? Which will be smarter?

## Step 3 — Try One Round by Hand

Before we build a 5-round game, we have to make sure our 'Game Master' (the computer) can handle a single round without getting confused. This is called a Hand Test.

```
In [ ]: if image_paths:
    p = random.choice(image_paths)
    show_image(p)

    player_guess = input(f"Guess one of these labels {classes}: ")
    ai_answer = answer_from_filename(p)

    correct = player_guess.strip().lower() == ai_answer.lower()
    print("Correct answer:", ai_answer)
    print(game_response(correct))

    if not correct:
        print(hint_for_label(ai_answer))
else:
    print("No images found.")
```

Run the test 3 times. Did the AI give you the same response every time? If not, our `random.choice` is working!

### What's Happening Under the Hood?

- The Random Pick: `random.choice(image_paths)` is the computer reaching into the bag of images and pulling one out with its eyes closed.
- The Comparison: `correct = player_guess... == ai_answer...` is the computer comparing two strings of text.
- Why we use `.strip().lower()`? It makes the computer "chill out." If the player types " Cat " with extra spaces or "CAT" in all caps, the computer won't get mad—it will still know they mean "cat".
- The Personality Check: `game_response(correct)` calls that "skill" we defined earlier to see if the AI says something fun or boring.

## Step 4 — Build the Full Game Loop

```
In [ ]: # 1. The Setup (The Pre-Game Lobby)
# Every game needs a starting line. We set our 'Power Level' (score) to
# zero and decide how long the match lasts. If we want a 'Boss Level,' we
# could change rounds to 10!
rounds = 5
score = 0

#2. The Random Selector (The Dealer)
# The Logic: random.sample(image_paths, rounds).
# This is like shuffling a deck of cards. Instead of playing the same
# images in the same order every time, the computer picks 5 random 'cards'
# from our game_assets folder so the game stays surprising.
if len(image_paths) >= rounds:
    chosen_images = random.sample(image_paths, rounds)
```

```

else:
    chosen_images = image_paths

# 3. The Loop (The Heartbeat)
# The Logic: for round_number, p in enumerate(...).
# This is the heartbeat of the game. Everything inside this loop happens
over and over again until the 5 rounds are finished. enumerate is just a
fancy way for the computer to keep track of which round we are on (Round 1,
Round 2, etc.).
for round_number, p in enumerate(chosen_images, start=1): ## The
Indentation (The "Hug"): Everything indented (pushed to the right) is part
of the loop. If you "un-indent" the scoring part, the game will only
calculate the score once at the very end!
    print("\n" + "=" * 40)    ## The Visual Cleanup: Notice the "\n" + "=" *
40. This is purely for the player. It draws a clear line between rounds so
the screen doesn't look like a giant wall of messy text.
    print(f"Round {round_number}")
    show_image(p)

    player_guess = input(f"Your guess {classes}: ") ## The input() Pause:
the computer "freezes" at the input line. It is waiting for the human to do
something. This is the User Interaction part of UX.
    ai_answer = answer_from_filename(p)

    correct = player_guess.strip().lower() == ai_answer.lower()

# 4. The Points (Risk vs. Reward)
# The Logic: score += 10 vs score -= 5.
# This makes the game competitive! You get a big reward for being
right, but a small penalty for being wrong. This is Game Balance—if the
penalty was -100, the game would be too frustrating!
    if correct:
        score += 10
    else:
        score -= 5

    print("Correct answer:", ai_answer)
    print(game_response(correct))
    print("Score:", score)

    if not correct:
        print(hint_for_label(ai_answer))

print("\n🎉 Game Over!")
print("Final Score:", score)

```

## Hack the Game

- Change the stakes: "Make a 'Hard Mode' where a wrong answer takes away 50 points."
- Change the length: "Make a 'Speed Run' that is only 2 rounds long."
- Personalize the end: "Change the 'Game Over' message to something specific to your personality."

## Step 5 — Add a Final Message

🏆 **The Concept: "The Victory Screen"** Every great game—from Roblox to Fortnite—gives the player a rank at the end. Without this code, the game just stops. This function looks at the final "XP" (score) and decides if the player is a Novice, an Expert, or a Master.

```
In [ ]: def final_score_message(score):  
        # The computer checks these from top to bottom.  
        if score >= 40: ## The "Thresholds" (The Numbers): Why did we pick 40  
            and 20? Since we have 5 rounds and each correct answer is worth 10 points,  
            a 'Perfect Score' is 50.  
            return "🏆 Amazing job! You are an AI Game Master!"  
        if score >= 20:  
            return "👏 Nice work! Your game is working well."  
        return "🌱 Good start! Keep improving your game." ## This is for  
        anyone who didn't hit 20 or 40 points. It's the "participation trophy" that  
        encourages them to try again.  
  
        print(final_score_message(score))
```

- If you changed the game to have 10 rounds, what should these numbers change to? (Answer: They should probably double!)
- What if we put the 'Good Start' check at the top? Everyone would get that message! We check for the highest achievement first, like a gold medal, then silver, then bronze.

## Customize their trophies.

Change the text and the emojis to fit their game's theme:

- Space Theme: "🏆 Galactic Commander" vs "🌱 Space Cadet"
- Cooking Theme: "🏆 Five-Star Chef" vs "🌱 Dishwasher"
- Zombie Theme: "🏆 Ultimate Survivor" vs "🌱 Zombie Bait"

## 🧠 Step 6 - The "AI Vision" Demo

Let's see how a real AI brain looks at your pictures. It doesn't read the filename; it looks at the pixels! A Classification Confidence score — the bread and butter of AI.

```
In [ ]: # --- THE "AI BRAIN" PREVIEW ---  
        print("🧠 AI is 'looking' at the pixels now...")
```



```

# In a real AI, we calculate these based on features.
# Let's simulate what the AI sees:
ai_confidence_cat = random.randint(70, 95)
ai_confidence_dog = 100 - ai_confidence_cat

print(f"📊 AI Analysis of the image:")
print(f"🐱 Cat Probability: {ai_confidence_cat}%")
print(f"🐶 Dog Probability: {ai_confidence_dog}%")

if ai_confidence_cat > ai_confidence_dog:
    print("AI says: 'I'm pretty sure this is a Cat!'")
else:
    print("AI says: 'This definitely looks like a Dog to me.'")

```

## Step 7 — Reflection

Today, you built the Machine. It loads images, asks questions, tracks scores, and gives rewards. Next class, we are going to give this machine a Brain (so it can see without filenames) and a Soul (so it talks with real personality)!

Answer these questions:

1. What part of your game worked best?
2. What would you improve next?
3. How did the AI brain and AI personality work together?

## Bonus Idea

Ask a partner to play your game and give feedback:

- Was it fun?
- Was it clear?
- Was it too easy or too hard?